

CSCI 423

Introduction to Algorithms



DR. RAAFAT ELFOULY

Talk Outline



- BIO
- Course Outline
- Grading scheme
- Text Book
- Introduction

BIO



- Dr. Raafat Elfouly...
- relfouly@ric.edu
- CL 244 Tel: 6-9758

Course outline



- Objectives: This course introduces the techniques used to analyze the time complexities of computer algorithms and surveys algorithms used to solve common programming problems.
- Topics include "big-oh" analysis, recursion, sorting algorithms, selection, graphs, priority queues and dynamic programming.

Grading:

- There will be two examinations, written homework and programming project. Each examination counts for 20 percent of your grade. The homework and programming project count for the remaining 60 percent.

Course outline



Attendance:

- Attendance and Active participation is expected for all classes and is required for examinations. The student is responsible for all material missed because of absence for any reason.
- No Late assignments, no makeup exams

Slides:

- Will be posted for one week only and then removed

Keys to success

- Course is fun, but intense
- Start early, work steadily, don't fall behind
- Course is accumulative, builds on earlier lessons – if you have questions, ask early and often!

Collaboration



- Collaboration is good – but you must be able to demonstrate that you yourself have the skills and knowledge required by the end of the course
- Think together, but solve/Code individually

Text Book



- Analysis of Algorithms an Active learning approach”, Jeffrey McConnel, Second edition, Jones and Bartlett ISBN 9780763707828.

Summary of Topics



- Introduction
- Big-Oh Analysis Basics
- Recursive Algorithms
- Searching and Selection Algorithms
- Analysis of Sorting Algorithms
- Graph Algorithms
- Dynamic Programming and Shortest Paths
- NP-Completeness
- Parallel Algorithms



Why Study Algorithms?

- important for all other branches of computer science
- plays a key role in modern technological innovation
 - “Everyone knows Moore’s Law – a prediction made in 1965 by Intel co-founder Gordon Moore that the density of transistors in integrated circuits would continue to double every 1 to 2 years....in many areas, performance gains due to improvements in algorithms have vastly exceeded even the dramatic performance gains due to increased processor speed.”
 - Excerpt from *Report to the President and Congress: Designing a Digital Future*, December 2010 (page 71).



Why Study Algorithms?

- important for all other branches of computer science
- plays a key role in modern technological innovation
- provides novel “lens” on processes outside of computer science and technology
- challenging (i.e., good for the brain!)
- fun



Skills You'll Learn

- Become a better programmer
- Sharpen your mathematical and analytical skills
- Start “thinking algorithmically”
- Literacy with computer science’s “greatest hits”
- Ace your technical interviews

Concept is very old!!!



- The word 'algorithm' has its roots in Latinizing the name of [Muhammad ibn Musa al-Khwarizmi](#) in a first step to *algorismus*.^{[11][12]} Al-Khwārizmī (c. 780–850) was a [Persian](#) mathematician, [astronomer](#), [geographer](#), and scholar in the [House of Wisdom](#) in [Baghdad](#), whose name means 'the native of [Khwarazm](#)', a region that was part of [Greater Iran](#) and is now in [Uzbekistan](#).^{[13][14]}
- <https://en.wikipedia.org/wiki/Algorithm>

What is an algorithm?

14

An algorithm is a sequence of **unambiguous** instructions for solving a problem, i.e., for obtaining a required **output** for any legitimate **input** in a **finite amount of time**.

How it differs from Computer program?

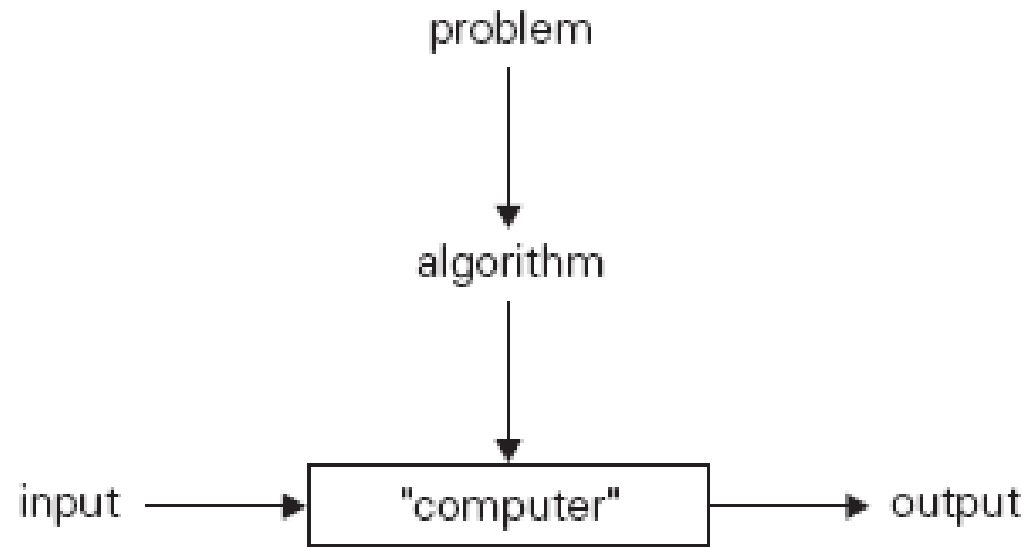


FIGURE 1.1 The notion of the algorithm.

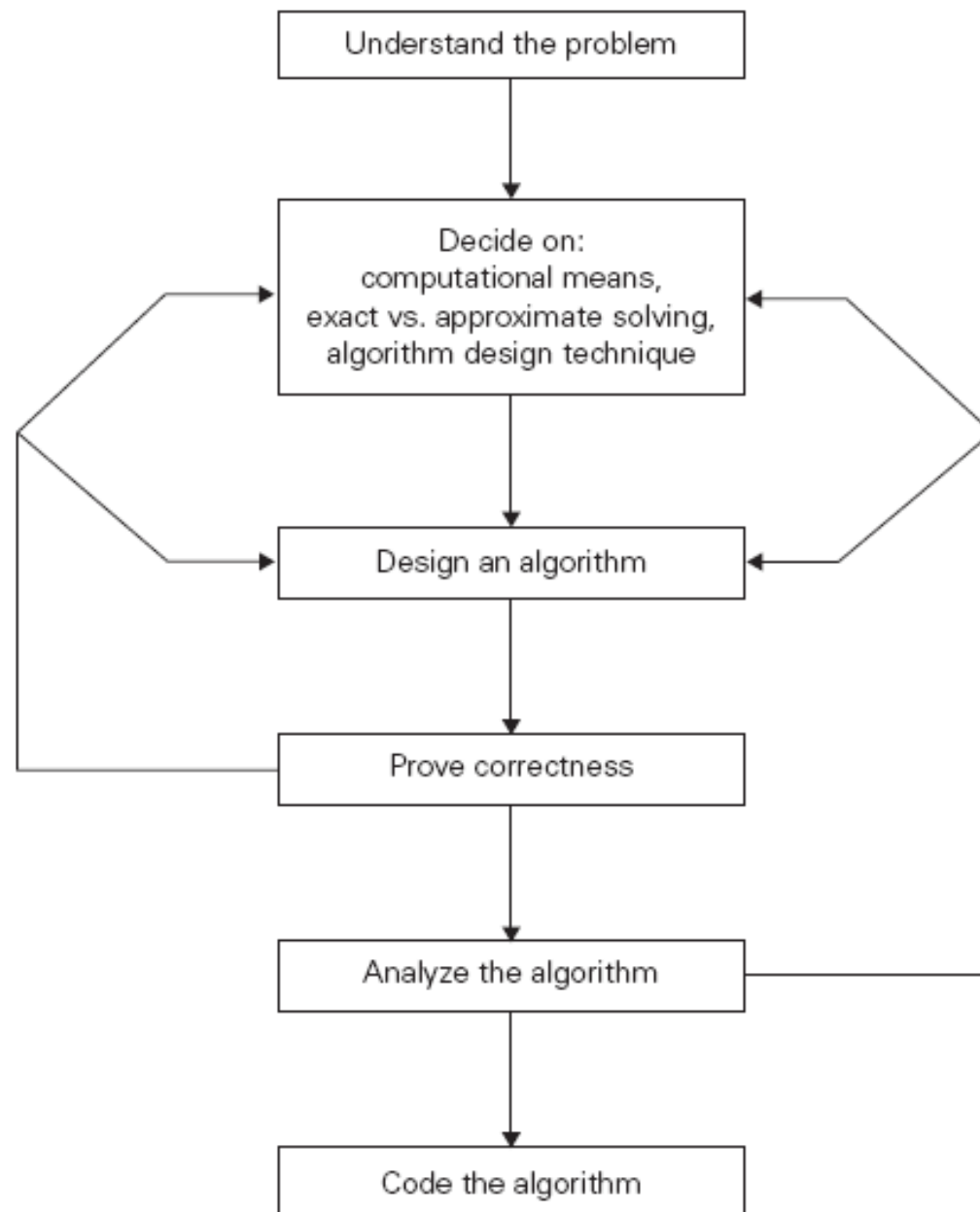


FIGURE 1.2 Algorithm design and analysis process.

Recall: Definition



- Computer science is a discipline that involves the understanding and design of computers and computational processes.

A well-educated computer scientist should be able to...



- apply the fundamental concepts and techniques of
 - computation,
 - algorithms, and
 - computer design
- to a specific problem

...including



- detailing of specifications,
- analysis of the problem,
- provide a design that
 - functions as desired,
 - has satisfactory performance,
 - is reliable and maintainable,
 - and meets desired cost criteria.



- Search Engines

Chapter One



- Algorithms and Data Structures
- Pseudocode
- Algorithm Features
- What is analysis?
- What to count and consider
- Mathematical background
- Rates of growth
- Analyzing programs

Prerequisites

22

- Before beginning this chapter, you should be able to:
 - Read and create algorithms
 - Identify comparison and arithmetic operations
 - Use basic algebra

Goals

23

- At the end of this chapter you should be able to:
 - Describe how to analyze an algorithm
 - Explain how to choose the operations that are counted and why others are not
 - Explain how to do a best-case, worst-case, and average case analysis

Goals (continued)

24

- Work with logarithms, probabilities, and summations
- Describe $\Theta(f)$, $\Omega(f)$, $O(f)$, growth rate, and algorithm order

Algorithms and Data Structures



- Algorithm - A recipe for performing a certain task.
 - Examples:....
- Data structure - A way of arranging data to make solving a particular problem easier.

Algorithm Features



- Correctness
- Maintainability
- Efficiency

Pseudocode



- Text that is a lot like a programming language but that is not really a programming language.
- Pseudocode (also called Program Design Language, PDL) is English-like statements that precisely describe specific operations¹:
 - Action statements
 - Focuses on the logic of the program
 - Avoids language-specific elements
 - Written at a level so that code can be generated almost automatically.

Pseudocode



Write a pseudocode to find the straightline distance between two points (x_1, y_1) and (x_2, y_2) ??

Pseudocode



Write a program to find the straight line distance between two points

1. Start
2. Declare variables: X_1 , Y_1 , X_2 , Y_2 , Distance
3. Prompt user to enter X_1 and Y_1
4. Prompt user to enter X_2 and Y_2
5. Calculate the straightline distance
6. Display Distance
7. Stop

Pseudocode



Write a pseudocode to find the average of three numbers??

Pseudocode



Write a pseudocode to find the minimum of three numbers

Analyzing an Algorithm

32

- Show that it properly solves the problem
- Determine how efficient it solves the problem
 - Approximate the number of operation
 - Memory requirements
 - Time needed to solve it

Algorithm Classification

33

- **Iterative**

- Based on loop and conditional statements
- Determine the work in the loop and the number of times the loop executes

- **Recursive**

- Based on calls to the same subprogram with a smaller problem
- Determine the number of smaller problems and the amount of work done to produce them and then put their solutions together

What is Analysis?

34

- An approximation of how long an algorithm will take or how much extra space it needs
- Comparison of two or more algorithms that solve the same problem
- Independent of the computer because a faster computer does not make an algorithm more efficient **Why?**
- Independent of initializations because those may be done faster

Space Complexity

35

- The amount of space needed for an algorithm to complete its task
- Algorithms are classified as either in place or needing extra space
 - Depends on whether the algorithm moves data around within its current storage or copies information to a new space while it's working
- Use of extra space can be critical in software designed for small embedded systems

What to Count

36

- Comparisons
 - Equal, greater, not equal, ...
- Arithmetic
 - Additions
 - ✦ add, subtract, increment, decrement
 - Multiplications
 - ✦ multiply, divide, modulus, remainder

Cases to Consider

37

- **Best Case**
 - The least amount of work done for any input set
- **Worst Case**
 - The most amount of work done for any input set
- **Average Case**
 - The amount of work done averaged over all of the possible input sets

Average Case

38

- Determining the average case:
 - Find the number of input set classes
 - Find the probability that the input will be from each of these classes
 - Find the amount of work done for each class
- The average case is given by:

$$A(n) = \sum_{i=1}^m p_i * t_i$$

Mathematical Background Review

39

- The floor of X is the largest integer $\leq$ to X
- The ceiling of X is the smallest integer $\geq$ to X
- The logarithm (base Y) of X is the value Z that satisfies the equation $X = Y^Z$
- *Examples*

Binary Trees

40

- A binary tree with N nodes has at least $\lfloor \lg N \rfloor + 1$ levels
- There are 2^{K-1} nodes on level K
- In a complete binary tree all of the nodes on levels 1 to $k-1$ have two children, so all of the leaves are on level k
- A complete binary tree with k levels has $2^k - 1$ nodes
- Example 3,4 level binary tree

Probabilities

41

- A probability is a number between 0 and 1
- If something will never occur, it has a probability of 0
- If something always occurs, it has a probability of 1
- If there are N equally likely events, each one has a probability of $1/N$
- *Example throwing an Die*
- *Coin*

Summations

42

- A summation is a compact way to express the sum of a series of values
- The sum of the numbers from 1 to 7 would be written:

$$\sum_{i=1}^7 i$$

- There are closed forms for many summations given in the text

Useful summation formulas and rules

43

$$\sum_{l \leq i \leq u} 1 = 1 + 1 + \cdots + 1 = u - l + 1$$

In particular, $\sum_{1 \leq i \leq n} 1 = n - 1 + 1 = n \in \Theta(n)$

$$\sum_{1 \leq i \leq n} i = 1 + 2 + \cdots + n = n(n+1)/2 \approx n^2/2 \in \Theta(n^2)$$

$$\sum_{1 \leq i \leq n} i^2 = 1^2 + 2^2 + \cdots + n^2 = n(n+1)(2n+1)/6 \approx n^3/3 \in \Theta(n^3)$$

$$\sum_{0 \leq i \leq n} a^i = 1 + a + \cdots + a^n = (a^{n+1} - 1)/(a - 1) \text{ for any } a \neq 1$$

In particular, $\sum_{0 \leq i \leq n} 2^i = 2^0 + 2^1 + \cdots + 2^n = 2^{n+1} - 1 \in \Theta(2^n)$

$$\sum (a_i \pm b_i) = \sum a_i \pm \sum b_i \quad \sum c a_i = c \sum a_i \quad \sum_{l \leq i \leq u} a_i = \sum_{l \leq i \leq m} a_i + \sum_{m+1 \leq i \leq u} a_i$$



Algorithm Efficiency and Big-O

Algorithm Efficiency and Big-O

- Getting a precise measure of the performance of an algorithm is difficult
- Big-O notation expresses the performance of an algorithm as a function of the number of items to be processed
- This permits algorithms to be compared for efficiency

Linear Growth Rate

- If processing time increases in proportion to the number of inputs n , the algorithm grows at a linear rate

```
public static int search(int[] x, int target) {  
    for(int i=0; i < x.length; i++) {  
        if (x[i]==target)  
            return i;  
    }  
    return -1; // target not found  
}
```

Linear Growth Rate

- If processing time is proportional to the number of inputs, the growth rate is linear

- If the target is not present, the for loop will execute `x.length` times
- If the target is present the for loop will execute (on average) $(x.length + 1) / 2$ times
- Therefore, the total execution time is directly proportional to `x.length`
- This is described as a growth rate of order n OR
- $O(n)$

```
public static int search(int[] x, int target) {  
    for(int i=0; i < x.length; i++) {  
        if (x[i]==target)  
            return i;  
    }  
    return -1; // target not found  
}
```

Quadratic Growth Rate

- If processing time is proportional to the square of the number of inputs n , the algorithm grows at a quadratic rate

```
public static boolean areUnique(int[] x) {  
    for(int i=0; i < x.length; i++) {  
        for(int j=0; j < x.length; j++) {  
            if (i != j && x[i] == x[j])  
                return false;  
        }  
    }  
    return true;  
}
```


Quadratic Growth Rate (cont.)

- If processing time is proportional to the number of inputs n ,

- The for loop with i as index will execute $x.length$ times
- The for loop with j as index will execute $x.length$ times
- The total number of times the inner loop will execute is $(x.length)^2$
- The growth rate has an order of n^2 or
- $O(n^2)$

```
public static boolean  
    for(int i=0; i < x.length; i++) {  
        for(int j=0; j < x.length; j++) {  
            if (i != j && x[i] == x[j])  
                return false;  
        }  
    }  
    return true;  
}
```

Big-O Notation

- The $O()$ in the previous examples can be thought of as an abbreviation of "order of magnitude"
- A simple way to determine the big-O notation of an algorithm is to look at the loops and to see whether the loops are nested
- Assuming a loop body consists only of simple statements,
 - ▣ a single loop is $O(n)$
 - ▣ a pair of nested loops is $O(n^2)$
 - ▣ a nested pair of loops inside another is $O(n^3)$
 - ▣ and so on . . .

Formal Definition of Big-O

- Consider the following program structure:

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        Simple Statement  
    }  
}  
  
for (int i = 0; i < n; i++) {  
    Simple Statement 1  
    Simple Statement 2  
    Simple Statement 3  
    Simple Statement 4  
    Simple Statement 5  
}  
  
Simple Statement 6  
Simple Statement 7  
...  
Simple Statement 30
```

Formal Definition of Big-O (cont.)

- Consider the following program structure:

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        Simple Statement  
    }  
}  
  
for (int i = 0; i < n; i++) {  
    Simple Statement 1  
    Simple Statement 2  
    Simple Statement 3  
    Simple Statement 4  
    Simple Statement 5  
}  
  
Simple Statement 6  
Simple Statement 7  
...  
Simple Statement 30
```

**This nested loop
executes a *Simple
Statement* n^2 times**

Formal Definition of Big-O (cont.)

- Consider the following program structure:

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        Simple Statement  
    }  
}
```

```
for (int i = 0; i < n; i++) {  
    Simple Statement 1  
    Simple Statement 2  
    Simple Statement 3  
    Simple Statement 4  
    Simple Statement 5  
}
```

Simple Statement 6

Simple Statement 7

...

Simple Statement 30

**This loop executes 5
Simple Statements n times
($5n$)**

Formal Definition of Big-O (cont.)

- Consider the following program structure:

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        Simple Statement  
    }  
}
```

```
for (int i = 0; i < n; i++) {  
    Simple Statement 1  
    Simple Statement 2  
    Simple Statement 3  
    Simple Statement 4  
    Simple Statement 5  
}
```

Simple Statement 6

Simple Statement 7

...

Simple Statement 30



**Finally, 25 Simple
Statements are executed**

Formal Definition of Big-O (cont.)

- Consider the following program structure:

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        Simple Statement  
    }  
}  
for (int i = 0; i < n; i++) {  
    Simple Statement 1  
    Simple Statement 2  
    Simple Statement 3  
    Simple Statement 4  
    Simple Statement 5  
}  
Simple Statement 6  
Simple Statement 7  
...  
Simple Statement 30
```

We can conclude that the relationship between processing time and n (the number of data items processed) is:

$$T(n) = n^2 + 5n + 25$$

Formal Definition of Big-O (cont.)

- In terms of $T(n)$,

$$T(n) = O(f(n))$$

- There exist

- ▣ two constants, n_0 and c , greater than zero, and
- ▣ a function, $f(n)$,

- such that for all $n > n_0$, $cf(n) = T(n)$
- In other words, as n gets sufficiently large (larger than n_0), there is some constant c for which the processing time will always be less than or equal to $cf(n)$
- $cf(n)$ is an upper bound on performance

Formal Definition of Big-O (cont.)

- The growth rate of $f(n)$ will be determined by the fastest growing term, which is the one with the largest exponent

- In the example, an algorithm of

$$O(n^2 + 5n + 25)$$

is more simply expressed as

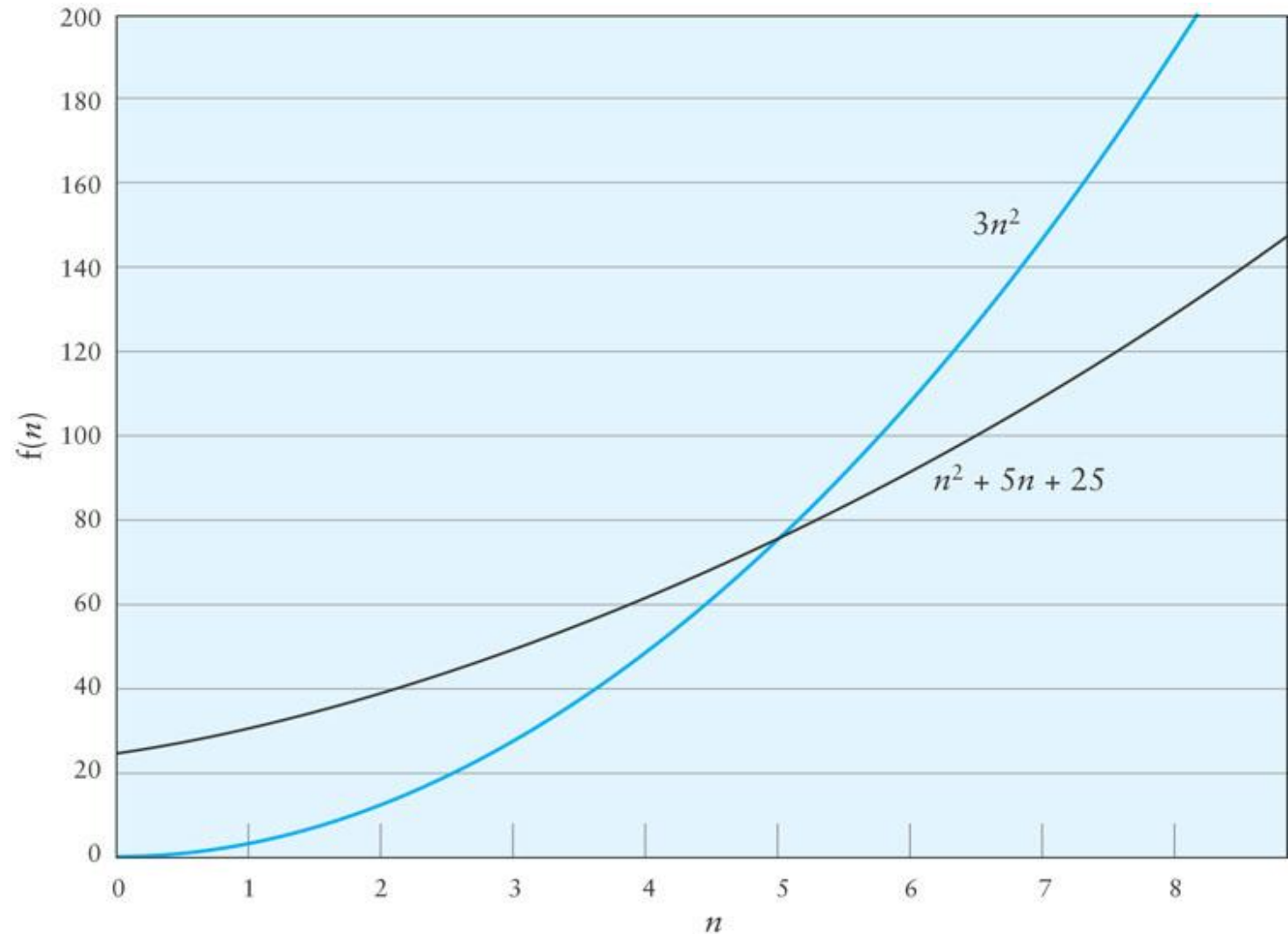
$$O(n^2)$$

- In general, it is safe to ignore all constants and to drop the lower-order terms when determining the order of magnitude

Big-O Example 1

- Given $T(n) = n^2 + 5n + 25$, show that this is $O(n^2)$
- Find constants n_0 and c so that, for all $n > n_0$, $cn^2 > n^2 + 5n + 25$
 - ▣ Find the point where $cn^2 = n^2 + 5n + 25$
 - ▣ Let $n = n_0$, and solve for c
$$c = 1 + 5/n_0 + 25/n_0^2$$
- When n_0 is $5(1 + 5/5 + 25/25)$, c is 3
- So, $3n^2 > n^2 + 5n + 25$ for all $n > 5$
- Other values of n_0 and c also work

Big-O Example 1 (cont.)



Big-O Example 2

- Consider the following loop

```
for (int i = 0; i < n; i++) {  
    for (int j = i + 1; j < n; j++) {  
        3 simple statements  
    }  
}
```

- $T(n) = 3(n - 1) + 3(n - 2) + \dots + 3$

- Factoring out the 3,

$$3(n - 1 + n - 2 + n - 3 + \dots + 1)$$

- $1 + 2 + \dots + n - 1 = (n \times (n-1))/2$

Big-O Example 2 (cont.)

- Therefore $T(n) = 1.5n^2 - 1.5n$
- When $n = 0$, the polynomial has the value 0
- For values of $n > 1$, $1.5n^2 > 1.5n^2 - 1.5n$
- Therefore $T(n)$ is $O(n^2)$ when n_0 is 1 and c is 1.5

Big-O Example 2 (cont.)

